Pendle / PT Looping

Aug 7, 2026

Table of Contents

Summary

Overview

Issues

- [WP-M1] swapBridgeWithdrawAll minimum-output check counts the box's pre-existing output-token and input-token balance
- [WP-M2] claimMMRewards's unrestricted external call lets a malicious/compromised frontend/backend drain the box and money-market position to an arbitrary address
- [WP-L3] Box owners should be allowed to terminateSession.
- [WP-L4] _boxBridgeToken() fails to validate that msg.value == b.nativeFee.
- [WP-L5] PendlePTLooper._boxSwapToken() returns incorrect values when tokenIn == tokenOut.
- [WP-L6] AaveMoneyMarketView._liquidity calculates borrowable liquidity using the physical balance of aTokens.
- [WP-L7] PendlePTLooper._boxSwapToken() fails to validate minSwapRate when tokenIn == tokenOut.
- [WP-L8] Rewards that are airdropped to MoneyMarket will be locked in the contract
- [WP-I9] Consider adding token out amount to the TokenSwapped event
- [WP-I10] Consider bypassing the _ltvCheck for deleveraging operations that don't involve a withdrawal
- [WP-I11] Owner's isZeroPriceImpact intent is not bound to the swap execution

Appendix

Disclaimer

Summary

This report has been prepared for Pendle / PT Looping smart contract, to discover issues and vulnerabilities in the source code of their Smart Contract as well as any contract dependencies that were not part of an officially recognized library. A comprehensive examination has been performed, utilizing Static Analysis and Manual Review techniques.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.

Overview

Project Summary

Project Name	**Pendle / PT Looping**
Codebase	**https://github.com/pendle-finance/pendle-core-v2**
Commit	**e4b9adc282c6599af39fbe0cf69ec18b9a31bdc4**
Language	**Solidity**

Audit Summary

Delivery Date	**Aug 7, 2026**
Audit Methodology	**Static Analysis, Manual Review**
Total Issues	**11**

[WP-M1] swapBridgeWithdrawAll minimum-output check counts the box's pre-existing output-token and input-token balance

Medium

Issue Description

`swapBridgeWithdrawAll` checks the owner's `minTokenOut` against `_boxSwapToken`'s return — the box's **full** `tokenOut` balance after the swap, not the swap's output. `_boxSwapToken` is also called with `minSwapRate = 0`, disabling its internal delta check. So the only absolute slippage floor the owner signs is broken: it can be satisfied by `tokenOut` the box already held, letting the proposer's swap spend the debt token for little or no new output.

Two independent ways the floor is weakened:

1. **Pre-existing `tokenOut` is counted** — `postBal` includes balance held before the swap, not just the swap delta.

2. **Swap input is read at execution time** — `tokenInBal = _boxBal(s.box, tokenIn)` is live, not the amount the owner assumed when signing, so the effective rate drifts looser/stricter regardless of (1).

Vulnerable Code

```
180  {
181   address tokenIn = _debtTokenHere(s.bd);
182  (postBal,) = _boxSwapToken(s.box, tokenIn, _boxBal(s.box, tokenIn),
     tokenOutHere, 0, ms.swap);
183  }
184   require(postBal >= req.minTokenOut, "PTLooper: insufficient token out");
```

At current HEAD `_boxSwapToken` already returns the correct value as a second return (`netTokenOut`), but `swapBridgeWithdrawAll` discards it with `(postBal,)` and checks the full balance.

Recommendation

Add a dedicated `minSwapRate` to `SwapBridgeWithdrawAllRequest` so swap slippage and final-amount floor are controlled separately.

Status

Acknowledged

[WP-M2] claimMMRewards's unrestricted external call lets a malicious/compromised frontend/backend drain the box and money-market position to an arbitrary address

Medium

Issue Description

The project's stated security invariant (Notion "PT-Looping System Overview", *Notable changes*) is: *"In case that proposer/executor became malicious, the PTLooper contract ensures that only messages constrained by the user signed request can be executed. User fund is safe under their signed request."*

`PendlePTLooper.claimMMRewards()` forwards the user-signed `req.extRewardController` and `req.data` straight into `BaseMoneyMarket.claimRewards()`, which executes `(bool success,) = extRewardController.call(data);` — an **unrestricted external call with no target allowlist and no calldata/selector validation**.

Root cause: that call executes in the context of the **per-box money-market clone** (-`NEEDS_ACCOUNT == true` for both Aave and Morpho, so `onBehalf == address(this)` = the clone). The clone (a) holds the entire Aave/Morpho position and (b) is granted **infinite ERC20 approvals by the box** for `spokePt` and `spokeDebtToken` at `LoopPositionBox.initialize()`. An arbitrary call running *as* the clone can therefore move funds to an attacker-chosen address — either immediately, or via a deferred allowance pulled in a later transaction:

- `extRewardController = spokeDebtToken` (or `spokePt`), `data = transferFrom(box, attacker, amount)` → drains the box's idle token balances (the clone holds the box's infinite approval).

- `extRewardController = MORPHO`, `data = borrow(marketParams, assets, 0, onBehalf=clone, receiver=attacker)` or `withdrawCollateral(marketParams, assets, onBehalf=clone, receiver=attacker)` (Aave `pool.borrow` / `pool.withdraw(... to=attacker)` equivalents) → borrows against / withdraws the collateral straight to the attacker.

- `extRewardController = aToken` (the clone's collateral is a transferable ERC20), `data = approve(attacker, type(uint256).max)` → grants the attacker a **standing allowance**; the actual `aToken.transferFrom(clone, attacker, …)` happens in a **separate later transaction**. This deferred variant defeats any in-call balance/position check (see Recommendation).

Who the attacker is (not necessarily the proposer): the malicious `extRewardController-/data` live in the **owner-signed** `ClaimMMRewardsRequest`. The on-chain `proposer` signs only `ClaimMMRewardsMessage{permit}`, which carries **no call data**. So the party able to plant the malicious call is **whoever constructs the EIP-712 request the user is asked to sign — the dApp frontend/backend** (compromisable via XSS, a poisoned/MITM'd frontend, or a supply-chain attack), *not* necessarily the trusted on-chain proposer. The attack succeeds **even with an honest proposer and executor**: they neither sign nor can inspect the opaque `data`, and the request is a valid owner signature, so to them it is an authorized "claim rewards". This widens the threat surface from "trusted proposer/executor" to "any compromise of the request-construction layer".

Violated invariant: a user-signed request is supposed to *constrain* fund movement; here it authorizes an arbitrary, unconstrainable transfer/approval. The `rewardTokens` before/after delta forwarded to `box.OWNER()` is **not** a safeguard — malicious `data` sends funds directly to `attacker` (or sets an allowance), bypassing the delta accounting entirely (the delta for the listed `rewardTokens` can be zero). `data` is opaque `bytes` a wallet cannot decode, so a "claim rewards" prompt becomes a blind-signed authorization to drain the whole position.

Worst-case impact: complete loss of the box's idle balances and of the money-market collateral-/borrowing power for any user who blind-signs a single crafted "claim rewards" request — funds end up at an attacker-chosen address.

(For context, not the headline: the internal consistency note recorded that at an earlier commit `57894124` `BaseMoneyMarket.claimRewards` had been *removed* in favor of a box-level `directBoxCall`; at the current HEAD it has been **re-introduced** in this unrestricted form and runs from the position-holding clone.)

```
 84   function claimRewards(
 85   MarketCtx memory ctx,
 86   address extRewardController,
 87   bytes calldata data,
 88   address[] memory rewardTokens
 89  ) external onlyPtLooper returns (uint256[] memory claimedAmount) {
 90  @@ 90,94 @@
 95
 96  (bool success,) = extRewardController.call(data);
 97   require(success, "MoneyMarket: claimRewards failed");
 98
 99  @@ 99,106 @@
107  }
```

```
287   function claimMMRewards(
288   ClaimMMRewardsMessage memory ms,
289   bytes memory proposerSig,
290   ClaimMMRewardsRequest memory req,
291   bytes memory ownerSigIfFirstCall
292  ) external onlyExecutor {
293  (Session memory s, bytes32 requestHash) = _ensureSessionAndAuth(ms, req,
     proposerSig, ownerSigIfFirstCall);
294   uint256[] memory claimedAmounts =
295   s.spokeMM.claimRewards(_marketCtx(s), req.extRewardController, req.data,
     req.rewardTokens);
296   emit RewardsClaimed(
297   ms.permit.owner, ms.permit.boxId, requestHash, ms.permit.nonce,
     req.rewardTokens, claimedAmounts
298  );
299  }
```

```
34   function initialize(
35   address _owner,
36   bytes32 _boxId,
37   address _spokeMoneyMarket,
38   address _ptToApprove,
39   address _debtTokenToApprove
40  ) external initializer {
41   OWNER = _owner;
42   BOX_ID = _boxId;
43   SPOKE_MONEY_MARKET = _spokeMoneyMarket;
44
45   if (_spokeMoneyMarket != address(0)) {
46   _safeApproveInf(_ptToApprove, _spokeMoneyMarket);
47   _safeApproveInf(_debtTokenToApprove, _spokeMoneyMarket);
48  }
49  }
```

```
112   struct ClaimMMRewardsRequest {
113   RequestHeader header;
114   address extRewardController;
115   bytes data;
116   address[] rewardTokens;
117  }
```

PoC

1.

A user wants to claim rewards. A **malicious or compromised frontend/backend** builds the `ClaimMMRewardsRequest` the user is asked to sign, with `extRewardController = spokeDebtToken`, `data = abi.encodeWithSelector(IERC20.transferFrom.selector, box, attacker, boxBalance)`, and `rewardTokens = []`.

2. The user blind-signs the EIP-712 payload — the wallet shows only an address plus an opaque `bytes` blob under a "claim rewards" label, with no way to decode that the blob is a `transferFrom` draining the box.

3. The (honest or malicious) proposer signs the trivial `ClaimMMRewardsMessage{permit}` — which contains no call data — and the executor calls `claimMMRewards`. Neither inspects `data`; the owner signature makes it look authorized.

4. Inside `claimRewards`, the **clone** executes `spokeDebtToken.transferFrom(box, attacker, boxBalance)` using the box→clone infinite approval → the box's idle balance is drained to `attacker`. With `rewardTokens = []` the delta accounting loops zero times, so nothing is detected.

5. Variants: `extRewardController = MORPHO`, `data = borrow(..., receiver=attacker)` / `withdrawCollateral(..., receiver=attacker)` drains the position directly; or `extRewardController = aToken`, `data = approve(attacker, max)` sets a standing allowance the attacker pulls in a **later** transaction (no funds move during the claim).

The snippet below is **illustrative** (not a runnable test) — it shows the crafted request and the resulting effect executed from the clone:

```
// --- Malicious/compromised frontend crafts the request the user is asked to "sign to claim rewards" ---
ClaimMMRewardsRequest memory req = ClaimMMRewardsRequest({
 header: someValidHeader,
 extRewardController: spokeDebtToken,          // any token / market, not a reward distributor
 data: abi.encodeWithSelector(                 // opaque to the wallet
 IERC20.transferFrom.selector,
 box,                                        // box has granted the clone infinite approval
 attacker,                                   // attacker-chosen receiver
 IERC20(spokeDebtToken).balanceOf(box)       // drain the whole idle balance
),
 rewardTokens: new address[](0)                // empty => before/after delta accounting is a no-op
```

```
});
// user blind-signs req; proposer signs ClaimMMRewardsMessage{permit} (no data); executor calls claimMMRewards(...)


// --- Effect inside BaseMoneyMarket.claimRewards, msg.sender == the per-box clone ---
// (bool success,) = extRewardController.call(data);
//   == spokeDebtToken.transferFrom(box, attacker, boxBalance)  // succeeds: clone holds box's inf approval


// --- Deferred variant that defeats any in-call invariant check: approve now, pull later ---
req.extRewardController = aToken;                // clone's collateral is a transferable ERC20
req.data = abi.encodeWithSelector(IERC20.approve.selector, attacker, type(uint256).max);
// claim tx moves nothing (balances/LTV unchanged); in a SEPARATE tx:
//   aToken.transferFrom(clone, attacker, aToken.balanceOf(clone));   // collateral stolen later
```

Recommendation

The clone both holds the money-market position and is the approved spender of the box's tokens, so it must **never** perform a caller-controlled external call. The weaker mitigations are **insufficient** — do not rely on them:

- *Forcing the in-call value recipient to box/owner* does nothing against `data = approve(attacker, max)` (e.g. on the clone's transferable aTokens, or re-approving the box's tokens to the attacker): the claim call transfers nothing, so there is no recipient to constrain.

- *Asserting balances / collateral / debt / LTV are unchanged after the call* also passes for the `approve` payload, because `approve` mutates no balance or position state. The attacker then drains in a **separate, later transaction** using the standing allowance.

Robust fix — remove the arbitrary `extRewardController`/`data` and claim through **typed, hard-coded interfaces** for an admin-registered set of distributors, building the calldata in-contract with the recipient/account pinned to the clone/box/owner:

```
// No caller-supplied calldata. Recipient/account are forced by the contract.
function claimAaveRewards(MarketCtx memory ctx, address[] calldata assets) external
onlyPtLooper {
 REWARDS_CONTROLLER.claimAllRewards(assets, ctx.box.OWNER());   // `to` forced to owner
}


function claimMerkl(MarketCtx memory ctx, address reward, uint256 amount, bytes32[]
calldata proof)
 external onlyPtLooper
{
 uint256 pre = _selfBalance(reward);
 MERKL_DISTRIBUTOR.claim(_one(address(this)), _one(reward), _one(amount), _one(proof));
// account = clone
 _transferOut(reward, ctx.box.OWNER(), _selfBalance(reward) - pre);                  //
sweep to owner
}
```

If a generic mechanism is truly unavoidable, allowlist **both** the target **and** the selector **and fully decode-and-validate every argument** — reject any allowance/operator/transfer or position-mutating selector outright (`approve`, `permit`, `increaseAllowance`, `setApprovalForAll`, `transfer`, `transferFrom`, and the money market's `borrow`/`withdraw`/`withdrawCollateral`/`supply`), and pin every address argument (recipient/account/spender) to the clone/box/owner. A target+selector allowlist **alone** is not enough: an allowed selector that can still set an allowance or name an arbitrary receiver re-opens the same drain (including the deferred `approve`-then-pull path).

Status

ⓘ Acknowledged

[WP-L3] Box owners should be allowed to terminateSession.

Low

Issue Description

The box owner might want to cancel the pending action earlier without relying on `executor`.

```
47   function terminateSession(address owner, bytes32 boxId, bytes32 requestHash)
    external onlyExecutor {
48   UserRequestInfo storage userReq = _userRequest[owner][boxId];
49   require(userReq.requestHash == requestHash, "PTLooper: requestHash mismatch");
50
51   uint32 newSessionNonce = ++userReq.sessionNonce;
52   delete userReq.requestHash;
53
54   emit SessionTerminated(owner, boxId, requestHash, newSessionNonce);
55  }
```

Recommendation

Allow the box `owner` to call `terminateSession` for their own `(owner, boxId)`, so users can always cancel if the backend is unavailable.

Status

✔ Fixed

[WP-L4] _boxBridgeToken() fails to validate that msg.value == b.nativeFee.

Low

Issue Description

If `msg.value > b.nativeFee`, the excess native tokens remain stranded in the `PendlePTLooper` contract.

https://github.com/pendle-finance/pendle-core-v2/blob/-6f650b20ab34799c2b55ab1e72385353ec96f428/contracts/pt-looping/PendlePTLooper.sol#L227--L238

```
227  function _boxBridgeToken(IPLoopPositionBox box, BridgeData memory b, address token,
     uint256 amount) internal {
228  amount = amount.min(b.maxAmount);
229  if (amount == 0) return;
230  box.approveAndCall{value: b.nativeFee}(
231  ApprovedCall({
232  token: token, amount: amount, approveTo: b.call.approve, callTo:
     b.call.extRouter, data: b.call.callData
233  }),
234  msg.sender
235  );
236
237  emit TokenBridged(box, token, amount);
238  }
```

```
172  function swapBridgeWithdrawAll(
173  SwapBridgeWithdrawAllMessage memory ms,
174  bytes memory proposerSig,
175  SwapBridgeWithdrawAllRequest memory req,
176  bytes memory ownerSigIfFirstCall
177  ) external payable onlyExecutor {
178  (Session memory s, bytes32 requestHash) = _ensureSessionAndAuth(ms, req,
     proposerSig, ownerSigIfFirstCall);
179
180  address tokenOut = _onHub(s.bd) ? req.hubTokenOut : req.dstTokenOut;
181
182  uint256 postBal;
183  {
184  address tokenIn = _debtTokenHere(s.bd);
```

```
185  postBal = _boxSwapToken(s.box, tokenIn, _boxBal(s.box, tokenIn), tokenOut,
     0, ms.swap);
186  }
187  require(postBal >= req.minTokenOut, "PTLooper: insufficient token out");
188
189  uint32 destChainId = req.receiveOnSpoke ? s.bd.spokeChainId : s.bd.hubChainId;
190  if (destChainId != block.chainid) {
191  _boxBridgeToken(s.box, ms.bridge, tokenOut, postBal);
192  } else {
193  _boxWithdraw(s.box, ms.permit.owner, tokenOut, postBal);
194  emit Withdrawn(ms.permit.owner, ms.permit.boxId, requestHash,
     ms.permit.nonce, tokenOut, postBal);
195  }
196  }
```

```
55  function approveAndCall(ApprovedCall memory call, address nativeRefund)
56  external
57  payable
58  onlyManager
59  returns (bytes memory result)
60  {
61  uint256 nativeAmount = call.token == NATIVE ? call.amount : 0;
62  uint256 nativeFee = msg.value;
63
64  uint256 initialNativeBalance = _selfBalance(NATIVE) - nativeAmount - nativeFee;
65
66  _approveForExtRouter(call.token, call.approveTo, call.amount);
67  result = call.callTo.functionCallWithValue(call.data, nativeAmount + nativeFee);
68  _approveForExtRouter(call.token, call.approveTo, 0);
69
70  uint256 finalNativeBalance = _selfBalance(NATIVE);
71
72  if (finalNativeBalance > initialNativeBalance) {
73  _transferOut(NATIVE, nativeRefund, finalNativeBalance -
     initialNativeBalance);
74  }
75  }
```

Status

ⓘ Acknowledged

[WP-L5] PendlePTLooper._boxSwapToken() returns incorrect values when tokenIn == tokenOut.

Low

Issue Description

When `tokenIn == tokenOut`, `PendlePTLooper._boxSwapToken()`:

- Expected return:
  - `postTokenOutBal`: `_boxBal(box, tokenOut)` (the total `tokenOut` amount in the box after the swap, including the pre-existing `tokenOut` balance).
  - `netTokenOut`: `tokenInBal` (the `tokenOut` increment corresponding to the `tokenIn` increment).

- Current implementation returns:
  - `postTokenOutBal`: `tokenInBal` (excludes the pre-existing `tokenOut` balance in the box).
  - `netTokenOut`: 0

https://github.com/pendle-finance/pendle-core-v2/blob/-43e97f2ce3f04281e988b848fb5ff4035db1d9f0/contracts/pt-looping/PendlePTLooper.sol#L214--L242

```
214   function _boxSwapToken(
215   IPLoopPositionBox box,
216   address tokenIn,
217   uint256 tokenInBal,
218   address tokenOut,
219   uint256 minSwapRate,
220   BoxApprovedCall memory swap
221  ) internal returns (uint256 postTokenOutBal, uint256 netTokenOut) {
222   if (tokenIn != tokenOut) {
223   uint256 preTokenOutBal = _boxBal(box, tokenOut);
224   box.approveAndCall(
225   ApprovedCall({
```

```
226   token: tokenIn,
227   amount: tokenInBal,
228   approveTo: swap.approve,
229   callTo: swap.extRouter,
230   data: swap.callData
231  }),
232   address(box)
233  );
234   postTokenOutBal = _boxBal(box, tokenOut);
235   netTokenOut = postTokenOutBal - preTokenOutBal;
236
237   require(netTokenOut * PMath.ONE >= tokenInBal * minSwapRate, "PTLooper:
     swap below minSwapRate");
238   emit TokenSwapped(box, tokenIn, tokenInBal, tokenOut, postTokenOutBal);
239  } else {
240   postTokenOutBal = tokenInBal;
241  }
242  }
```

Status

ⓘ Acknowledged

[WP-L6] AaveMoneyMarketView._liquidity calculates borrowable liquidity using the physical balance of aTokens.

Low

Issue Description

Since Aave v3.1+ introduced virtual accounting, it should use `getVirtualUnderlyingBalance`. The current `borrowingLiquidity` is overestimated, especially for the GHO branch.

`_liquidity` calculates available liquidity using the **physical balance** (`_balanceOf-`) of aTokens. However, Aave v3.1+ enables virtual accounting; the true borrowing limit is `virtualUnderlyingBalance` (as donations are excluded). Using physical balance overestimates `borrowingLiquidity`. Consequently, relayers/frontends may set an inflated `amountDebtBorrowed`, causing on-chain `borrow` calls to revert due to insufficient virtual balance (no loss of funds).

The impact is more severe for the GHO branch: aGHO is now a standard pooled asset (-`ATokenInstance`, `getIsVirtualAccActive = true`), but the code still relies on a hardcoded `maxBorrowable` exception. This currently results in an over-reporting of approximately 34% (~15M GHO).

```
121   if (address(bd.spokeDebtToken) == GHO_ASSET) {
122   borrowing = maxBorrowable;
123  } else {
124  (address aTokenAddr,,) = dp.getReserveTokensAddresses(bd.spokeDebtToken);
125   borrowing = PMath.min(maxBorrowable, _balanceOf(aTokenAddr, bd.spokeDebtToken));
126  }
```

The Aave v3.1+ borrow path caps liquidity using `reserve.virtualUnderlyingBalance -= liquidityTaken` (with uint128 checked subtraction) and has `usingVirtualBalance` hardcoded to true, with no special GHO branch.

Impact

- Impacts view functions/off-chain logic only; **no loss of funds**. Overestimation may cause on-chain `borrow` reverts.

- Deviations for standard assets are negligible (~0.003%), triggering only when a reserve's liquidity is nearly exhausted.

- The GHO-specific exception leads to an over-reporting of ~34%, representing the most significant deviation.

Recommendation

Standardize the calculation to use `min(maxBorrowable, getVirtualUnderlyingBalance)` and remove the GHO-specific exception:

```
uint256 byLiq = dp.getVirtualUnderlyingBalance(bd.spokeDebtToken);   // Replace _balanceOf(aTokenAddr, …)
borrowing = PMath.min(maxBorrowable, byLiq);
```

1. The `IPool` / `IPoolDataProviderV3` interfaces in the repo lack the declaration for `getVirtualUnderlyingBalance`; this must be added (use the DataProvider version which returns uint256).

2. The GHO-specific exception should be removed. If it must be kept for Aave-fork compatibility, the GHO branch must still be updated to use `min(maxBorrowable, getVirtualUnderlyingBalance(GHO))`.

Status

✔ Fixed

[WP-L7] PendlePTLooper._boxSwapToken() fails to validate minSwapRate when tokenIn == tokenOut.

Low

Issue Description

There is no `require(PMath.ONE >= minSwapRate, "...");` check when `tokenIn == tokenOut`.

https://github.com/pendle-finance/pendle-core-v2/blob/-43e97f2ce3f04281e988b848fb5ff4035db1d9f0/contracts/pt-looping/PendlePTLooper.sol#L214--L242

```
214   function _boxSwapToken(
215   IPLoopPositionBox box,
216   address tokenIn,
217   uint256 tokenInBal,
218   address tokenOut,
219   uint256 minSwapRate,
220   BoxApprovedCall memory swap
221  ) internal returns (uint256 postTokenOutBal, uint256 netTokenOut) {
222   if (tokenIn != tokenOut) {
223   uint256 preTokenOutBal = _boxBal(box, tokenOut);
224   box.approveAndCall(
225   ApprovedCall({
226   token: tokenIn,
227   amount: tokenInBal,
228   approveTo: swap.approve,
229   callTo: swap.extRouter,
230   data: swap.callData
231  }),
232   address(box)
233  );
234   postTokenOutBal = _boxBal(box, tokenOut);
235   netTokenOut = postTokenOutBal - preTokenOutBal;
236
237   require(netTokenOut * PMath.ONE >= tokenInBal * minSwapRate, "PTLooper:
     swap below minSwapRate");
238   emit TokenSwapped(box, tokenIn, tokenInBal, tokenOut, postTokenOutBal);
239  } else {
240   postTokenOutBal = tokenInBal;
241  }
242  }
```

Status

Acknowledged

[WP-L8] Rewards that are airdropped to MoneyMarket will be locked in the contract

Low

Issue Description

The current `MoneyMarket` implementation only supports `BaseMoneyMarket.claimRewards()` at L100, which executes `extRewardController.call(data);`. This mechanism strictly requires the `MoneyMarket` contract to actively initiate calls to the `extRewardController` to claim rewards.

There is no apparent method to recover assets that are pushed or claimed on the contract's behalf. This includes, but is not limited to, the following scenarios:

- Rewards claimed on behalf of the `MoneyMarket` contract via the Morpho Universal Rewards Distributor (used with the Market Rewards Program Registry), where UniversalRewardsDistributor.claim() allows "***Anyone*** can claim rewards on behalf of an account."

- Rewards claimed on behalf of the `MoneyMarket` contract by an operator / governor / guardian via the AngleProtocol `Distributor.claim()`.

- Tokens airdropped directly to the `MoneyMarket` contract address.

```
108   /// @notice Claims rewards.
109   /// @param account The address to claim rewards for.
110   /// @param reward The address of the reward token.
111   /// @param claimable The overall claimable amount of token rewards.
112   /// @param proof The merkle proof that validates this claim.
113   /// @return amount The amount of reward token claimed.
114   /// @dev Anyone can claim rewards on behalf of an account.
115   function claim(address account, address reward, uint256 claimable, bytes32[]
      calldata proof)
116   external
117   returns (uint256 amount)
118  {
119   require(root != bytes32(0), ErrorsLib.ROOT_NOT_SET);
120   require(
121   MerkleProof.verifyCalldata(
122   proof, root, keccak256(bytes.concat(keccak256(abi.encode(account,
      reward, claimable))))
```

```
123  ),
124   ErrorsLib.INVALID_PROOF_OR_EXPIRED
125  );
126
127   require(claimable > claimed[account][reward], ErrorsLib.CLAIMABLE_TOO_LOW);
128
129   amount = claimable - claimed[account][reward];
130
131   claimed[account][reward] = claimable;
132
133   IERC20(reward).safeTransfer(account, amount);
134
135   emit EventsLib.Claimed(account, reward, amount);
136  }
```

```
412  @@ 412,420 @@
421   function _claim(
422   address[] calldata users,
423   address[] calldata tokens,
424   uint256[] calldata amounts,
425   bytes32[][] calldata proofs,
426   address[] memory recipients,
427   bytes[] memory datas
428  ) internal nonReentrant {
429  @@ 429,437 @@
438
439   for (uint256 i; i < usersLength; ) {
440   address user = users[i];
441   address token = tokens[i];
442   uint256 amount = amounts[i];
443   bytes memory data = datas[i];
444
445   // Only approved operators can claim for `user`
446   if (
447   msg.sender != user &&
448   tx.origin != user &&
449   mainOperators[msg.sender][token] == 0 &&
450   mainOperators[msg.sender][address(0)] == 0 &&
451   operators[user][msg.sender] == 0 &&
452   operators[user][address(0)] == 0 &&
453   !accessControlManager.isGovernorOrGuardian(msg.sender)
454  ) revert Errors.NotWhitelisted();
455
```

```
456  // Verifying proof
457  bytes32 leaf = keccak256(abi.encode(user, token, amount));
458  if (!_verifyProof(leaf, proofs[i])) revert Errors.InvalidProof();
459
460  // Closing reentrancy gate here
461  uint256 toSend = amount - claimed[user][token].amount;
462  claimed[user][token] = Claim(SafeCast.toUint208(amount), uint48(block-
     .timestamp), getMerkleRoot());
463  emit Claimed(user, token, toSend);
464
465  address recipient = recipients[i];
466  // Only `msg.sender` can set a different recipient for itself within the
     context of a call to claim
467  // The recipient set in the context of the call to `claim` can override
     the default recipient set by the user
468  if (msg.sender != user || recipient == address(0)) {
469  address userSetRecipient = claimRecipient[user][token];
470  if (userSetRecipient == address(0)) userSetRecipient =
     claimRecipient[user][address(0)];
471  if (userSetRecipient == address(0)) recipient = user;
472  else recipient = userSetRecipient;
473  }
474
475  if (toSend != 0) {
476  IERC20(token).safeTransfer(recipient, toSend);
477  if (data.length != 0) {
478  bytes32 callbackSuccess = IClaimRecipient(recipient).onClaim(user,
     token, toSend, data);
479  if (callbackSuccess != CALLBACK_SUCCESS) revert
     Errors.InvalidReturnMessage();
480  }
481  }
482  unchecked {
483  ++i;
484  }
485  }
486  }
```

```
88  function claimRewards(
89  MarketCtx memory ctx,
90  address extRewardController,
91  bytes calldata data,
92  address[] memory rewardTokens
93  ) external onlyPtLooper returns (uint256[] memory claimedAmount) {
94  uint256 n = rewardTokens.length;
```

```
95   uint256[] memory preBalance = new uint256[](n);
96   for (uint256 i = 0; i < n; ++i) {
97   preBalance[i] = _selfBalance(rewardTokens[i]);
98  }
99
100 (bool success,) = extRewardController.call(data);
101  require(success, "MoneyMarket: claimRewards failed");
102
103  address owner = ctx.box.OWNER();
104  claimedAmount = new uint256[](n);
105  for (uint256 i = 0; i < n; i++) {
106  claimedAmount[i] = _selfBalance(rewardTokens[i]) - preBalance[i];
107  _transferOut(rewardTokens[i], owner, claimedAmount[i]);
108 }
109
110  emit RewardsClaimed(extRewardController, rewardTokens, claimedAmount);
111 }
```

Status

ⓘ Acknowledged

[WP-I9] Consider adding token out amount to the TokenSwapped event

Informational

Issue Description

The `TokenSwapped` event should include the token out amount (or replace `postTokenOutBalance` with it) to better align with the `netIn` amount.

Currently, the `TokenSwapped` event only logs `postTokenOutBalance`, which is related to the swap result but easily prone to misinterpretation.

https://github.com/pendle-finance/pendle-core-v2/blob/-6f650b20ab34799c2b55ab1e72385353ec96f428/contracts/interfaces/IPPTLooper.sol#L133-L139

```
133   event TokenSwapped(
134   IPLoopPositionBox indexed box,
135   address indexed tokenIn,
136   uint256 netIn,
137   address indexed tokenOut,
138   uint256 postTokenOutBalance
139  );
```

https://github.com/pendle-finance/pendle-core-v2/blob/-6f650b20ab34799c2b55ab1e72385353ec96f428/contracts/pt-looping/PendlePTLooper.sol#L198--L225

```
198   function _boxSwapToken(
199   IPLoopPositionBox box,
200   address tokenIn,
201   uint256 tokenInBal,
202   address tokenOut,
203   uint256 minSwapRate,
204   BoxApprovedCall memory swap
205  ) internal returns (uint256 postTokenOutBal) {
206   if (tokenIn != tokenOut) {
207   uint256 preTokenOutBal = _boxBal(box, tokenOut);
208   box.approveAndCall(
209  @@ 209,216 @@
217  );
218   postTokenOutBal = _boxBal(box, tokenOut);
```

```
219   uint256 netTokenOut = postTokenOutBal - preTokenOutBal;
220   require(netTokenOut * PMath.ONE >= tokenInBal * minSwapRate, "PTLooper:
      swap below minSwapRate");
221   emit TokenSwapped(box, tokenIn, tokenInBal, tokenOut, postTokenOutBal);
222 } else {
223   postTokenOutBal = tokenInBal;
224 }
225 }
```

Status

✓ Fixed

[WP-I10] Consider bypassing the _ltvCheck for deleveraging operations that don't involve a withdrawal

Informational

Issue Description

A user may want to improve their health factor to avoid liquidation, but they cannot satisfy the `_ltvCheck`.

Deleveraging might be blocked by the LTV constraint. Oracle price fluctuations between off-chain signing and on-chain execution can cause the LTV check to revert.

Since a partial repayment only improves the user's position health, skipping the check is worth considering.

```
294   function _repayWithdrawWithLTV(
295   DecreasePositionMessage memory ms,
296   DecreasePositionRequest memory req,
297   Session memory s,
298   uint256 debtIn
299  ) internal {
300   MarketCtx memory ctx = _marketCtx(s);
301  (uint256 netRepaid, uint256 mmDebtBal) = s.spokeMM.repay(ctx, debtIn,
     ms.minDebtRepaid);
302  (uint256 netWithdrawn, uint256 mmPtBal) =
303   s.spokeMM.withdraw(ctx, mmDebtBal > req.targetMaxDebt ? ms.amountPtWithdrawn
     : 0, ms.minPtWithdrawn);
304   uint256 rate = s.spokeMM.oraclePtDebtRate(ctx);
305   _ltvCheck(mmPtBal, mmDebtBal, rate, req.postWithdrawMaxLTV,
     ms.postWithdrawMaxLTV);
306   emit MoneyMarketRepaidThenWithdrawn(s.box, netRepaid, netWithdrawn);
307  }
```

```
264 function _ltvCheck(
265  uint256 mmPtBal,
266  uint256 mmDebtBal,
267  uint256 mmOraclePtDebtRate,
268  uint256 ownerMaxLTV,
269  uint256 proposerMaxLTV
270 ) internal pure {
271  uint256 maxLTV = PMath.min(ownerMaxLTV, proposerMaxLTV);
272  require(mmDebtBal <= mmPtBal.mulDown(mmOraclePtDebtRate).mulDown(maxLTV),
    "PTLooper: maxLTV exceeded");
273 }
```

Status

ⓘ Acknowledged

[WP-I11] Owner's isZeroPriceImpact intent is not bound to the swap execution

Informational

Issue Description

`isZeroPriceImpact` is an **owner-signed**, session-constant flag. Its documented meaning is *"the swap is a 1:1 PT+YT mint via SY; the co-minted YT is withdrawn to the owner"* — implemented as a fixed YT withdrawal of `netPtSwappedOut` (the PT delta):

```
79   preInBalance = _boxBal(s.box, tokenIn);
80   _boxWithdraw(s.box, TREASURY, tokenIn, ms.fee);
81   uint256 netIn = preInBalance - ms.fee;
82
83   uint256 netPtSwappedOut;
84  (netPtToMM, netPtSwappedOut) = _boxSwapToken(s.box, tokenIn, netIn, s.bd.hubPt,
    minSwapRate, ms.swap);
85   if (req.isZeroPriceImpact) {
86   _boxWithdraw(s.box, ms.permit.owner, IPPrincipalToken(s.bd.hubPt).YT(),
    netPtSwappedOut);
87  }
88
89   if (_isCrossChain(s.bd)) {
90   _boxBridgeToken(s, ms.bridge, s.bd.hubPt, netPtToMM, s.bd.spokePt,
    req.header.swapBridgeSlippage, false);
91  }
```

Two gaps follow, both because the flag is decoupled from the actual swap:

1. The mode is only softly enforced. The swap route is proposer-controlled (`ms.swap`); `_boxSwapToken` does not allowlist the router or check it went through SY — the only guard is `minSwapRate`. So `isZeroPriceImpact = true` is enforced only by the YT-withdrawal reverting, and only when the box holds no stray YT: if `preYT >= netPtSwappedOut`, a non-mint swap does **not** revert and pre-existing box YT is simply handed to the owner, silently ignoring the intent.

2. The mode cannot be scoped per iteration. The first iteration swaps `req.initTokenIn` (which may be SY — where a mint is natural); subsequent iterations swap `hubDebtToken` (a plain swap, no YT). A single session-level flag governs all of them, so it is impossible to enable the mode for only the first iteration (or only the subsequent ones).

Recommendation

Make it per-iteration: either split the field like `initTokenIn` (`initIsZeroPriceImpact` for the first iteration, `isZeroPriceImpact` for the rest, kept owner-signed), or move `isZeroPriceImpact` into the proposer-signed message.

Status

ⓘ Acknowledged

Appendix

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by WatchPug; however, WatchPug does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication.

Disclaimer

This report is based on the scope of materials and documentation provided for a limited review at the time provided. Results may not be complete nor inclusive of all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Smart Contract technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. A report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on the reports in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, we disclaim all warranties, expressed or implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. We do not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.